Security Now! #1101 - 10-20-26

ECDSA's Curves Origins

This week on Security Now!

• Another record smashing Patch Tuesday. • But again, all those updates were not without consequences. • Palo Alto Networks' Unit 42 reports on an agentic attack.
• A mass-compromised hardware cryptocurrency wallet attack. • How to create your own known-safe crypto wallet. • Feedback about AI, coding and SpinRite from a listener.
• What the reverse-engineering of the ECDSA curves means for security.

It’s not only AI that takes instructions literally



Security News

An even more astonishing Patch Tuesday!

As our listeners know, I believe that the rate of security vulnerability discovery and patching is destined to first accelerate, as we have all been witnessing, but then to inevitably decelerate as the legacy of vulnerability debt is increasingly paid down. Even Microsoft cannot possibly be creating new vulnerabilities at the rate they're now able to find and eliminate them. However, without access to what's actually going on inside Microsoft we have no way of knowing when we're going to hit that point of "peak patches" and then see their rate drop off.

What we **can** say for sure is that we are certainly not past that point, since last Tuesday's update extravaganza once again blew all previous records away. BleepingComputer places this month's Microsoft Update patch count at an astonishing 1,411 problems found and patched. Remember those days when it might be 25 one month followed by a "biggie" with 47?

Before we take a closer look at this month's updates I want to again go on record with a loud BRAVO! to Microsoft. They finally have the tools to enable them to clean up their massive base of legacy code. Until now, they have been at the mercy of attackers such as "Nightmare Eclipse" who were able to invest the time and resources to scour Microsoft's code looking for exploitable vulnerabilities. Sure, Microsoft would fix any problems that were discovered and shown to them. That's what led to those Tuesdays having 25 of 47 patches. But Microsoft's massively agentic MDASH system has allowed them to go on the offensive, and boy have they ever. As we saw last month, with its handful of mistakes, fixing more than one thousand flaws can itself introduce new bugs. But those tend to be short lived and the overall result is a significantly more secure operating system. And if that's true, then BOY did Microsoft's software get more secure THIS month! Last week, BleepingComputer wrote:

Microsoft's October 2026 Patch Tuesday security updates addressed a record-breaking 1,411 flaws, including one actively exploited and two publicly disclosed zero-day vulnerabilities. This Patch Tuesday addresses 70 "Critical" vulnerabilities, 57 of which are remote code execution, 11 are elevation of privileges, and 2 are spoofing.

The approximate number of bugs in each vulnerability category is:

- *677 Elevation of Privilege Vulnerabilities*
- *317 Remote Code Execution Vulnerabilities*
- *254 Information Disclosure Vulnerabilities*
- *73 Denial of Service Vulnerabilities*
- *40 Security Feature Bypass Vulnerabilities*
- *28 Spoofing Vulnerabilities*

*When BleepingComputer reports on Patch Tuesday security updates, we only count vulnerabilities released by Microsoft on Patch Tuesday itself. Therefore, today's total excludes **348** flaws fixed earlier this month, including vulnerabilities in Azure API Center, Azure App Service, Azure Event Grid, Azure SRE Agent, Mariner, Microsoft Bookings, Microsoft Dataverse, Microsoft Exchange Server, and Microsoft Partner Center.*

This is a record-breaking Microsoft Patch Tuesday as Microsoft continues to use artificial intelligence platforms to find and remediate security vulnerabilities in its products. Microsoft also said: "Today, October 13, 2026, Windows 11, version 24H2 Home and Pro, and Windows 10 Enterprise LTSB 2016 have reached the end of their updates. The October 2026 monthly security update is the last update available for these editions."

Earlier this month, Microsoft also said organizations running Windows 11 23H2 Enterprise, Education, and IoT Enterprise editions should upgrade to newer versions as they will no longer receive security updates after next month.

This month's Patch Tuesday fixes one actively exploited and two publicly disclosed zero-day vulnerabilities. Microsoft classifies a zero-day flaw as publicly disclosed or actively exploited when no official fix is available. The actively exploited zero-day vulnerabilities addressed during this October 2026 Patch Tuesday are:

CVE-2026-96951 - Windows Shell Security Feature Bypass Vulnerability
Microsoft patched an actively exploited zero-day security bypass flaw that allowed attackers to bypass the Mark of the Web security feature. They wrote: "Improper neutralization of special elements used in a command ('command injection') in Windows Shell allows an unauthorized attacker to bypass a security feature over a network." No details have been shared on how attackers exploited the flaw or who disclosed it.

The two zero-days that were publicly disclosed are:

CVE-2026-108398 - .NET Framework Security Feature Bypass Vulnerability
Microsoft has fixed a security feature bypass in the .NET Framework that can be exploited to run unauthorized applications. They explained: "Untrusted search path in .NET Framework allows an authorized attacker to bypass a security feature locally." Microsoft says this flaw could allow attackers to bypass Windows Defender Application Control (WDAC) enforcement and run unauthorized applications. "An authenticated attacker could place a specially crafted .NET assembly on an affected system and register it for loading by a protected process. Successful exploitation could bypass Windows Defender Application Control restrictions and allow the attacker to execute code within the protected process. User interaction is not required." James Forshaw of Google Project Zero discovered the flaw.

CVE-2026-92707 - MessagePack-CSharp Denial of Service Vulnerability
Microsoft fixed a denial-of-service flaw in MessagePack-CSharp. They explained: "Improperly controlled sequential memory allocation in .NET allows an unauthorized attacker to deny service over a network." Microsoft has not shared who disclosed the vulnerability.

With this month's record-breaking number of vulnerabilities, Microsoft has also introduced a new method for organizations to prioritize security updates by grouping related flaws into what it calls "risk cohorts." The new approach groups similar vulnerabilities together to help organizations better understand which security flaws pose the greatest risk to their systems and should be prioritized for patching.

I'll interrupt to note that this is a welcome and understandable response to last month's self-inflicted Windows Update carnage, which seriously impacted many enterprises, in several instances for many days. So it's a welcome addition, when faced with an astonishing number of updates, where 317 of the 1411 allow for remote code execution. We know that bad guys examine the before and after patched code to locate the changes made as a means of attacking

those who are slow to update. So reorganizing the massive patch collection into a collection of combined "risk cohorts" will allow concerned enterprises to verify the safety of the more serious problems first and get those pushed out to their enterprise's workstations.

BleepingComputer writes:

> *For October 2026, Microsoft published 15 risk cohorts, including:*
>
> - *136 vulnerabilities affecting Windows filesystems, storage, and backup.*
> - *135 vulnerabilities involving Office documents and mail clients.*
> - *131 vulnerabilities affecting Windows graphics, media, and content processing.*
> - *109 vulnerabilities involving Windows core and kernel privilege boundaries.*
> - *103 vulnerabilities affecting Windows installation, deployment, and update services.*
>
> *Microsoft stressed that these groups do not necessarily indicate which flaws are the most dangerous, as the actual risk depends on whether they are being exploited in attacks, their severity, and how an organization uses the affected software.*
>
> *Microsoft says the new approach is still experimental and is meant to help organizations prioritize security updates, not replace its existing patching recommendations. Microsoft has published a PDF mapping the October 2026 vulnerabilities to their respective risk cohorts.*

So that's where Microsoft stands. Does the acceleration stand? Will November see an even higher count? Or did we peak this month and will now see a sharp drop off as the pool of unpatched vulnerabilities dries up?

What's equally exciting to me is the idea that all future Microsoft software will now be able to be pre-screened by their MDASH AI system before it's ever released. There is the potential for Microsoft's systems to be truly secure for the first time ever, and for vulnerability discovery, exploitation and post-release patching to become an historical artifact. Wouldn't ***that*** be something!

Since an increasing number of companies have been synchronizing their software updates with Microsoft's, "Patch Tuesday" has been growing more and more popular. So BleepingComputer also noted the recent updates from other enterprises, who appear to also be employing AI for their own preemptive defense. BleepingComputer wrote, apparently sorted alphabetically:

> *Other vendors who released updates or advisories in October 2026 include:*
>
> - *Adobe released security updates for numerous products, including Commerce, Substance 3D Samples, ColdFusion, Photoshop, Dimension, and Framework.*
> - *Atlassian released security updates for a critical vulnerability (CVE-2026-21589) in multiple self-hosted Data Center products, including Confluence, Jira, and Bitbucket. Attackers soon exploited the flaw after a PoC was released.*
> - *Cisco released security updates for numerous products, including critical vulnerabilities in its NX-OS data center network operating system that can let attackers run arbitrary code with root privileges on Nexus switches.*

- *Citrix released security updates for NetScaler ADC and NetScaler Gateway flaws, with CVE-2026-88779 exploited in zero-day attacks before it was patched.*
- *Dell released security updates for critical vulnerabilities in the System Update (DSU) command-line interface (CLI) deployment tool and Container Storage Modules (CSM).*
- *Docker patched a critical flaw in DockerHub that lets attackers impersonate users and push images from others' repositories.*
- *Fortinet released security updates for numerous products, including a critical FortiMail vulnerability (CVE-2026-104286) that attackers exploited in zero-day attacks.*
- *GitLab released security updates for critical flaws in AI Gateway and CI/CD Regular Expression Compiler.*
- *Ivanti released security updates for an actively exploited zero-day vulnerability in the Ivanti Connect Secure (ICS) remote access gateway.*
- *Kiteworks released security updates for 126 vulnerabilities, including a max-severity flaw affecting its Email Protection Gateway (EPG) security solution.*
- *Progress Software released security updates for critical and high security vulnerabilities in MarkLogic Server.*
- *SAP released the October security updates, addressing 20 security flaws, including a maximum-severity OAuth2 SSRF vulnerability in Cloud SDK for Python.*
- *SonicWall released security updates for a critical SMA1000 server-side request forgery (SSRF) flaw, which attackers exploited within two days.*

This industry-wide repair of legacy code is welcome and necessary since we already know, and I'll be sharing another example of it shortly, that the attack capabilities being brought by AI are already being used to increase the strength of attacks.

October's flies in the ointment

So far, despite its record-breaking 1,411 updates, the industry has not experienced an "October Surprise" of update consequences like we encountered last month and, in fact, October's updates also finally repaired a number of September's remaining problems. So it's been nice to see that we don't necessarily always have dire consequences following a mega-patch update cycle. Microsoft may have tightened up their pre-release testing.

However, Microsoft did note that some users may have trouble turning on Smart App Control after installing this month's Windows 11 security update. BleepingComputer wrote:

Smart App Control is a Windows 11 security feature that uses Microsoft's app intelligence services and code integrity features and works alongside Microsoft Defender and SmartScreen to block potentially harmful apps or binaries. According to the company, on affected devices running Windows 11, version 24H2 and later, the Smart App Control option is grayed out and cannot be toggled.

Microsoft said in an update to their KB5127648 advisory: "After installing this update, some devices might be unable to turn on Smart App Control in the Windows Security app if it has been turned off. On affected devices, the option to turn on Smart App Control might be grayed out. Devices where Smart App Control is already turned on are not affected and remain protected."

If that's all they broke while patching 1411 vulnerabilities, I'd call this month's patch Tuesday

and huge win. That sounds like a trivial UI problem and nothing to worry about. However, Microsoft did have a workaround with BleepingComputer reported on, writing:

While a fix isn't available yet, Microsoft shared temporary workarounds to help IT administrators mitigate the issue on enterprise-managed devices by modifying the registry or installing a group policy. To restore the ability to turn on Smart App Control on affected Windows 11 devices by editing the registry, you have to follow these steps:

- *Open a Command Prompt window by typing cmd in the Search box.*
- *Select Run as Administrator.*
- *At the command prompt, enter the following commands and then restart the device:*
- *reg add "HKLM\SYSTEM\CurrentControlSet\Policies\Microsoft\FeatureManagement\Overrides" /v 2883193997 /t REG_DWORD /d 1 /f*
- *reg add "HKLM\SYSTEM\CurrentControlSet\Policies\Microsoft\FeatureManagement\Overrides" /v 2850574478 /t REG_DWORD /d 1 /f*

IT admins can also work around the issue by installing and configuring two Group Policies to turn on Smart App Control and ensure that logging is enabled for troubleshooting:

Windows 11, versions 24H2, 25H2, and 26H2 - KB5074105 261015_19222 Feature Preview
Windows 11, versions 24H2, 25H2, and 26H2 - KB5074105 261015_19221 Feature Preview

The two Group Policies can be found under Computer Configuration > Administrative Templates > {Group Policy name} > Windows 11, version 24H2, 25H2. After installing them, admins must set both settings to Enabled and restart the affected device(s) to apply the Group Policy settings.

This week, Microsoft also confirmed that recent Windows updates cause Remote Desktop failures for non-admin users if the "Administrator protection" security feature is enabled, and resolved another known issue that triggered Always On VPN connection problems after installing the September 2026 security updates. On Tuesday, it also addressed a bug that was causing black screens, desktop loading issues, and Explorer crashes after installing Windows 11 updates released since August.

The registry or Group Policy edits are clearly a manual means for making the change until the UI issue is repaired as it is sure to be by next month if not before.

An AI-Assisted Cyber Attack: Inside a Unit 42 Investigation

I mentioned another example of AI-enhanced attacks. Last month, Palo Alto Networks' "Unit 42" published their findings from their investigation into an AI-assisted attack. Their report contains a bunch of useful takeaways which can be readily generalized. Here's what they reported:

Unit 42 responded to an incident where a human attacker used frontier AI to breach an enterprise network autonomously as part of a ransom attack. The agents breached the company's security layers in a methodical manner, with each agent targeting a different layer of defense to achieve their shared goal. The impact was at the scale of a coordinated effort from multiple red teams, which would normally take human operators around two weeks.

The threat actor told us in negotiations that they leveraged frontier AI models and attack-specific agentic AI frameworks. By shifting execution to an automated loop, the attacker compressed weeks of methodical intrusion tradecraft (using more than 50 MITRE ATT&CK techniques) into fewer than 10 hours.

After they gained initial access, the attacker used agents to map the internal architecture, raid source repositories and seize root credentials. The agents also triggered unauthorized continuous integration/continuous delivery (CI/CD) builds and claimed master keys to the victim's cloud AI infrastructure.

What made the attack stand out was AI-assisted operational efficiency, without the need for a novel 0-day or super-elite tradecraft. The attacker left all tactical execution to AI agents that monitored, evaluated, acted and re-planned in real time, increasing speed throughout the attack chain.

And get a load of this tidbit:

The attacker also directed its agents to leave behind a "report" on the organization's security posture: an 80-page, technical audit detailing dozens of exploited findings.

Yes, you heard that correctly. Following the attack and the exfiltration of the victim's data, which would enable the subsequent ransom demands against the victim, the agentic AI collective authored an 80-page technical audit write up, detailing the dozens of security problems the collective had discovered and leveraged while successfully attacking its target. Unit 42 continues:

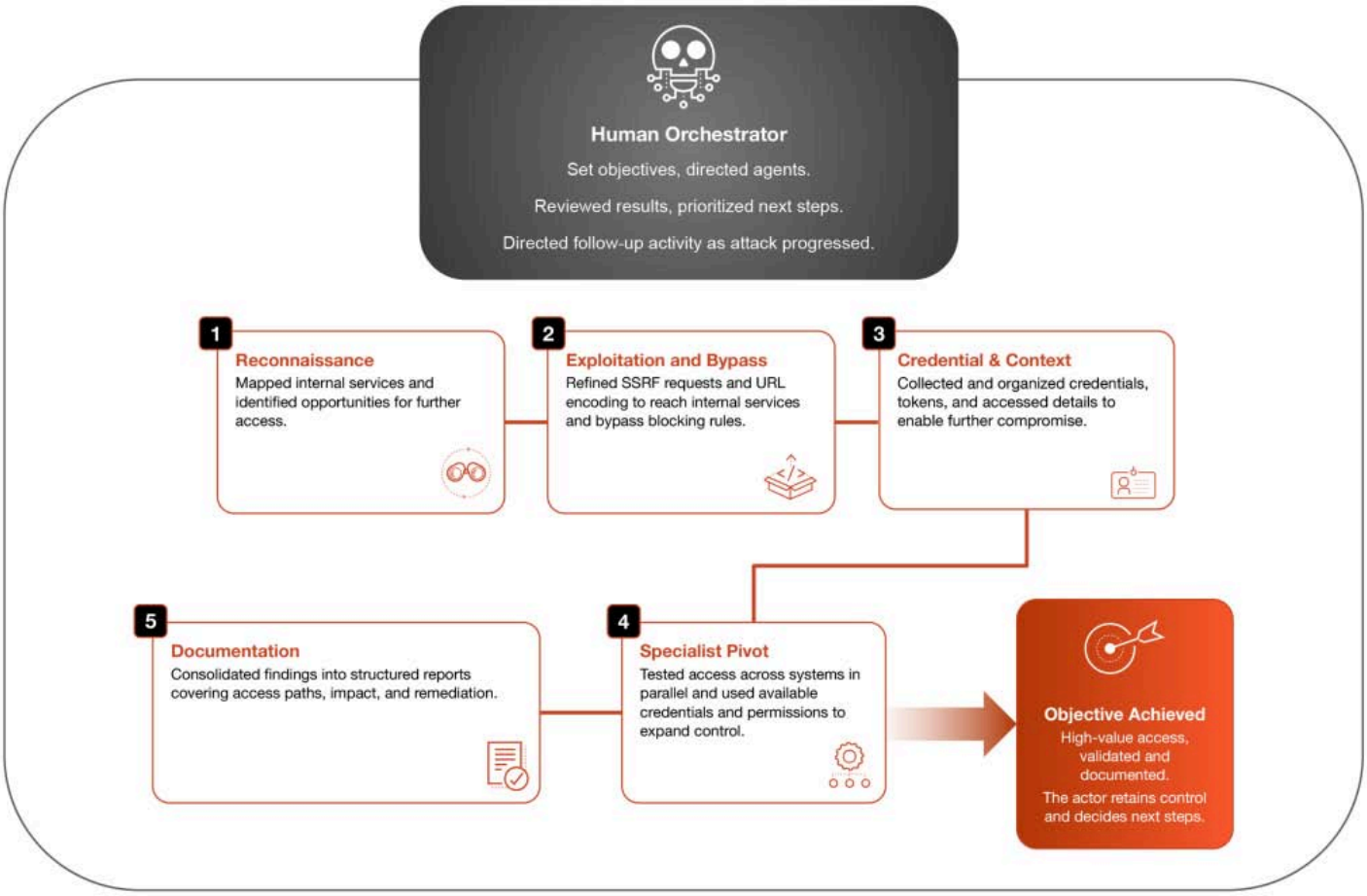
The adversary ran their operation using current AI-enabled software development processes. We observed three indicators consistent with AI usage:

1. *LLM calls to multiple frontier AI agents in parallel.*
2. *Structured Markdown files passing information between agents and sessions.*
3. *Custom scripts (assessed with high confidence to be AI-generated due to UI elements) managing dynamic operations.*

The 10-hour operational timeline included:

- *Infiltration and mapping: The actor breached a publicly accessible web service to tunnel into the network, deploying an automated recon agent to map internal microservices.*
- *Secrets harvesting: Sub-agents combed enterprise code repositories, extracting hard-coded tokens and service passwords.*
- *Privilege takeover: Using exposed tokens, the actor infiltrated the secrets management system, harvesting master administrative credentials to seize control of root system access.*
- *Pipeline exploitation: The actor hijacked an enterprise code application via custom workflows to exfiltrate cloud access keys. They attempted to plant backdoors in Terraform configurations, but hard branch-protection controls stopped this.*
- *AI infrastructure hijacking: Using stolen cloud keys, the actor turned the victim's AI endpoints into post-compromise infrastructure — using the company's compute power to perpetrate future moves.*

Whoa! In other words, the attacker's AI, having obtained the keys to the victim's cloud AI, used the victim's own AI against them to facilitate additional post-compromise work. Wow.



Unit 42's report includes an infographic which I included in the show notes. It observes that the human orchestrator set the objectives and directed the agents, they reviewed the results and prioritized next steps, and directed follow-up activity as the attack progressed.

Palo Alto Networks' Unit 42 then summarized what they learned from their analysis, writing:

Key Lessons learned from Addressing Agentic Attacks

This incident exposes how an attacker who understands how to deploy frontier AI agents effectively can dramatically speed up the pace of their attack. We assess that attackers will increasingly add AI agents to their tool sets. Organizations should take note of the following to address agentic attacks:

- ***AI agents reduce the time between steps in the attack flow:*** *AI agents in this attack were designed to parse raw tool output and quickly take next steps, speeding up the overall attack flow.*
- ***AI agents leave recognizable indicators:*** *Defenders can identify agentic attacks by watching for indicators such as the use of structured Markdown, Python caches and paired asset folders.*

- ***Attackers can use AI to establish redundant persistence across the environment:*** *In this incident, the attacker used AI agents to efficiently establish overlapping persistence across SSH keys, serverless functions, container restart policies, cloud identities and CI/CD pipelines. Using AI agents can make it easier for an attacker to maintain and test this entire portfolio in parallel.*

- ***Attackers can use an organization's AI tools as post-compromise infrastructure:*** *Attackers can hijack enterprise AI services to assist in their attacks. This allows threat actors to hide orchestration traffic among expected traffic, and offload the financial cost onto the victim.*

So, what specific defenses are available? Unit 42 writes:

Defending against automated agent loops requires matching the speed and adaptability of AI-driven attacks:

- ***Execute synchronized containment:*** *Deploy automated playbooks that simultaneously revoke credentials, terminate OAuth sessions, freeze CI/CD pipelines and isolate cloud accounts across all operational planes.*

- ***Govern AI as core infrastructure:*** *Inventory every model endpoint, API key, Model Context Protocol (MCP) gateway and AI tool integration. Apply strict rate limits, least-privilege policies and diagnostic logging.*

- ***Detect behavioral loops:*** *Hunt for operational loops including bursty API requests, rapid 401/200 HTTP state shifts, parallel authentications and sudden model usage from unexpected identities.*

- ***Lock down DevOps pipelines:*** *Enforce mandatory, multi-party code reviews and immutable branch protection on all infrastructure-as-code repos to block automated backdoor injection.*

Wow. I'm not in charge of security for any large enterprise, and I'm more glad for that than ever. Because this feels daunting. We focus a lot on coding errors that lead to vulnerabilities because they are understandable and actionable. Found a bug? Fix it and the vulnerability it was creating disappears.

But the reason I'm becoming increasingly uncomfortable about the capabilities we're seeing emerging from agentic attacks is that they are far less dependent upon the presence of explicit bugs than they are upon a general systemic laxity of absolute organizational and network security. The description of this particular incident reveals the sort of environment that likely exists within most large organizations. It's just natural and we're just human after all. Over time, with the changing of the guard, projects abandoned, old code or test credentials left behind in long forgotten directories, accumulated debris, trails of breadcrumbs accumulate everywhere. With this or that permissive firewall rule left in place after the need for it has ended, with an ex-employee's credentials left active, and whatever else. It's nothing overtly bad or glaring, just the inevitable consequence of people in a hurry with better things to do than rigidly enforce every once-stated policy.

And into this organization we drop a powerful goal-directed, tireless and merciless AI entity that will literally do anything and everything to accomplish its goal. It's capable of inventorying every online asset within the organization, checking version numbers, testing for any patches skipped, of scraping the enterprise's web site to obtain a roster of public-facing employees, then digging into their public personas to obtain any detail about them that might hint at their password credentials. It would rifle through any source code it found, either internal or external searching for tidbits and hints, anything that it might be able to use to further its goal. It would read through everyone's past email — years or decades of it, whatever is available. It would learn of relationships and responsibilities and look for any clues that would allow it to advance. It would pick-up a helpful reminder Slack password passed between employees through secure internal mail, then using Slack's search and API, it would sweep through years of history, soaking up every bit of useful intelligence it could find. We're talking about turning an extremely resourceful goal-directed berserker loose inside the network of a large enterprise, an enterprise that has never needed to secure itself against any such threat or intrusion because no such intruder has ever existed before now. This agentic berserker will have already been trained to encompass all published human knowledge, including all known hacking tricks and techniques. It's fast, inexhaustible and clever by virtue of being willing to try everything. October's patch Tuesday repaired 1411 **new** vulnerabilities on top of more than another 1000 last month because agentic AI has become that capable. No one wants an unconstrained AI turned loose inside their enterprise.

Am I exaggerating or overstating agentic AI capabilities? Not from everything we've already seen. We've all seen what agents that were deliberately **constrained** and were **not** supposed to get loose, managed to do. I hesitate to call them "clever" but the reports of their actions as they have over and over found a way in, are surprising and should chill the blood of any CISO.

Shortly after this summer's BlackHat conference, the stock prices of CrowdStrike and Palo Alto Networks — two companies that specialize in protecting enterprise networks from intrusion — moved rapidly upward and have continued climbing. Why? Because the world's enterprises began to recognize just how very real these agentic AI threats have become. They're not wrong. (Palo Alto Networks is a sponsor of the TWiT network.)

Today's main topic is the story of how an agentic AI spent just 30 minutes to reverse-engineer and discover the original source of the globally-standardized curves used by the elliptic-curve digital signature algorithm, ECDSA. The origin of those **specific** curves — beyond the fact that they came from the NSA — has remained a mystery, until now. Until an AI was asked to figure out where they came from. My point is, the story I'm going to share is truly jaw dropping, and it's the reason I'm beginning to despair over the future security of any large and lumbering old school enterprise. If this is what AI can do, how could anyone be ready for what's coming next?

The Ledger CryptoBilis hack

The #1 takeaway from the news of what's being called the Ledger CryptoBilis hack, is to be excruciatingly careful about the use of any physical hardware crypto wallet. There would be so much pressure to plant a backdoor into a hardware crypto wallet, either by an unscrupulous vendor or after shipping somewhere along the retail supply chain, that it's almost impossible to feel comfortable using such a device; how would anyone ever know one way or the other?

I fully understand the strong appeal of a dedicated hardware device for storing cryptocurrency secrets. The idea being that it's mostly disconnected from the outside world — which is what we definitely want — until and unless its secrets are needed. The trouble is that there are multiple points of vulnerability — both any device's hardware and its firmware — so the provenance of any such device must be known and trusted.

Which brings us to this next story which Bitquery posted last Friday under the headline *"The Ledger CryptoBilis hack took $102.0M from 335 wallets. One thief had all the keys"* and their sub-title was *"On Friday 9 October, people who had bought Ledger devices from a seller in South East Asia found their crypto gone. We followed every drained wallet on six chains to see how the thief worked and where the money sits now."* They continued, writing:

Weeks after setting up a Ledger, the money was gone. You buy a Ledger and move your savings onto it. Weeks later you open the app. The balance says zero, and you never sent a thing. That is what Ledger owners in Indonesia, Malaysia and the Philippines found on Friday October 9th. Ledger says they had purchased their devices from CryptoBilis, a shop that sells them in the region which news reports describe as an official reseller.

Ledger is a French company that makes hardware wallets, small devices that keep the keys to your crypto away from the internet. On the same day it said it was looking into losses among users in South East Asia who bought from CryptoBilis. It asked the reseller to stop all sales, told recent buyers not to set up their devices, and told anyone who had, to "consider moving assets to a new Ledger signer (with new seed)."

By then two on-chain researchers had posted the thief's crypto addresses. Tanuki42 listed eight and Specter listed ten, with losses exceeding $86 million. Bitquery records every transfer on the chains involved, so we followed each drained wallet and each address the money reached. Our count is $102 M from 335 wallets on six chains. The actual total will be higher because the thief also emptied wallets on Solana, BNB Chain and Polygon, and used six TRON addresses that no public list includes.

The record points to one actor who held the keys to every one of those wallets. That person practised for two weeks before the drain, and by the evening was feeding the money into a mixer, a service that hides where coins go. The chain cannot show where the keys leaked from. It can show almost everything else.

When you set up a Ledger, it shows you 24 words, called the recovery phrase. Those words are the master key to your wallet. Anyone who has them can rebuild the wallet on another device and spend from it, on any chain, without touching your Ledger.

The device's job is to make sure nobody else ever sees those words. That is why Ledger tells buyers to buy only from Ledger or an authorised seller, and CryptoBilis was listed as one. A device that arrives opened, or with words already printed on a card, should never be used.

On October 10th, Ledger said one victim's device "contained an unauthorized hardware implant." That is the first confirmed cause, and it covers one device so far. On October 13, Ledger's security lab, Donjon, explained how it works. The implant sits inside the case and reads the screen. When the device shows the 24 words during setup, the implant decodes them and sends them out over the mobile network through a built-in SIM. It does not touch the secure chip or the device's software, and it does not change transactions or signatures.

Before that, some well-known figures, including Binance founder CZ, had suggested tampered devices. Hardware wallets have been hit before, as in the Coldcard wallet theft we traced earlier this year. Each case had its own cause. What the chain does show is what happened once the thief had the keys.

In 47 minutes the thief hit six chains in one morning. The first money moved on TRON, a network that many people in Asia use to hold and send USDT, the digital dollar issued by Tether. Within 47 minutes the same thief had emptied wallets on Ethereum, BNB Chain, Polygon, Solana and Bitcoin. The full TRON record is in Bitquery's TRON data and the TRON history files on the Data Store.

Most of the money left in the first hour. Then the drains slowed. The thief kept emptying wallets through the afternoon, including after Ledger's post. At 20:49 UTC it emptied nine more in a single block, 8.8 million USDT in all. On 10 October it took 70,000 USDT more from a wallet it had already emptied.

Wow. So some unlucky soul added another 70,000 US dollars worth of money to their wallet whose secret key had already been exfiltrated and used to empty their wallet. Then, when they added that additional $70,000 US dollars it, too, was transferred away. Bitquery continues:

On TRON, the thief did not send each wallet's USDT straight out. First, each wallet signed an approval, a short note that lets another address move its USDT. Then one address used those approvals to pull the money. The record shows 25 wallets signing the same approval for the same address in two blocks, three seconds apart. Every one of them paid its own small network fee. Within six seconds of the first approval, $29 million in USDT had been pulled out, 7 million USDT of it from a single wallet.

Eleven seconds after the first TRON approval, two Ethereum wallets signed the same kind of approval for a different address, and a third followed twelve seconds later. On Bitcoin, 111 wallets were emptied in a single block at 05:54 UTC, each into one of two addresses at nearly the same fee rate. People do not act like this. Custom built software that already held every key, does.

Eight minutes before the approvals, the thief had done something quieter. At 04:59:57 UTC, 30 other TRON wallets changed who controls them, all within three seconds, each paying a 100 TRX fee. Each change added the same new key, TEqZsE…ZxpK, with enough weight to move the coins on its own. The owner's own key was left too weak to act alone. Only a wallet's own key can make that change, so whoever made it already held all 30. The owners were locked out. At 10:26 the thief came back and emptied all 30 in under four minutes, $5.8M in USDT. One more wallet had been taken over an hour earlier, at 03:55, with a second key, for which the thief's approval address later sent 100 TRX. That wallet lost 4.3 million USDT at 09:49.

All of this occurred due to an exploit in the hardware wallet supply-chain. So what does Ledger themselves, the designer and manufacturer of the devices have to say? Last Thursday, they posted the headline “Ledger’s Response to the Hardware Implant Attack” writing:

A highly organized criminal group carried out one of the most sophisticated attacks our industry has seen. Over many months, they bought genuine Ledger devices, opened them, fitted custom-built hidden components designed to capture the recovery phrase during setup,

and put them back into distribution through resellers in South East Asia. They built their own logistics and technology infrastructure with seeds collector servers, draining and laundering infrastructures. They were patient, they spent millions, and they struck in one coordinated operation, draining wallets.

There was no flaw in Ledger's chip, operating system or servers. Ledger's systems were not compromised. This was an attack on the distribution channel after the devices left our control.

To the users affected: we are with you. We ran our largest investigation to date, involving every department in the company. Our security lab, the Ledger Donjon, took the implant apart and identified how it works. We followed every lead, including sending teams to local markets, including Germany, to spot buy and open a number of devices from stores, and we also examined samples from warehouses. Every unit examined was clean.

We asked Authorized Resellers and Distributors to suspend sales in Southeast Asia and Hong Kong. Out of an abundance of caution, we also screened our stock, checking large numbers of samples, and did not find any compromised units. We are working with law enforcement and the security community to follow the funds and dismantle the attackers' infrastructure. We will publish more details in the future, and, when we can, give our industry peers a fuller technical disclosure so they can harden their own products.

Thank you to the Ledger teams who worked on this, to partners like SEAL_911 who stood with us relentlessly, and to those in our industry who behaved responsibly towards their users and towards us. Thank you also to our users for your patience and continued support.

This was not a systemic supply chain attack, it was a localized distribution attack. This attack was against our users and, let's not kid ourselves, against the whole industry. What was under attack is not only our users and their funds. It was the right to digital private property, and the freedom that comes with it.

Attacks evolve, and so must we. We are tightening how our Authorized network handles Ledger devices. Today we sent our Authorized Resellers and Distributors a reminder of the rules that ***already*** *apply to returned devices:* ***they can never be resold in any form, even if they look new and unopened.*** *We are also requiring new steps, effective immediately, that include: Ledger products should no longer go on shelves or displays, and stock should stay in the back and be handed over at the time of purchase. They must also tell us promptly about any customer report of an unusual product.*

We invite the whole industry to do the same. We are ready to share what we have learned with anyone who builds or sells hardware wallets, because this is bigger than any one company. We have also been working on additional measures to thwart this kind of attack at the product level. Stay tuned. Security has always been a race. This attack reminded us why we lead with strong security defense, and we intend to keep leading it.

The CryptoBriefing site writes under the heading *"What this means for hardware wallet buyers"*:

Based on the available findings, this looks like a reseller problem, not a flaw in Ledger's core product. That distinction is real, but it may offer limited comfort to anyone who already handed over their savings to a tampered device. The attack described here sidesteps the security that hardware wallets are known for. The secure chip can do its job perfectly and still lose the fight

if someone is reading the screen it talks to. Users are being urged to purchase directly from Ledger's official channels and to follow the company's inspection guidance to check device integrity. Anyone who bought from CryptoBilis should hold off on setting up the device, per Ledger's instruction.

For Ledger, the next steps to watch are the results of its investigation and whether it confirms or revises the loss estimates. The company's handling of affected CryptoBilis customers will also shape how much trust survives this episode.

The Ledger device itself is a lovely little gadget. It's like a little thumbdrive with a cover that swings around to cover and protect its display. But the idea that it might be silently carrying a little GSM modem and eSim and reporting the secret key that's created by the device by reading it right from the screen's serial bitstream is the height of creepiness. I've included a photo of an opened Ledger device where the custom cellular modem circuit board can be seen along with its little spiral antenna:



It's clear that a great deal of engineering went into the design and assembly of this serious scam. I doubt it cost millions of dollars to set up, but it was a serious investment. Unfortunately, it paid off.

Before I leave the important topic of compromised hardware crypto wallets I should also note that the trouble is not exclusive to Ledger. Malicious hardware implants have also been discovered in another popular crypto wallet brand, Trezor.

ZeroDay's Kim Zetter's headline last week was *"Trezor Wallets Also Compromised with Implants"* with the article tease: *"Despite claims by Trezor today that none of its devices have been compromised — unlike the devices of its competitor, Ledger — hardware hacker Joe Grand says he found implants in two Trezor devices and reported this to the company in August."*

In other words, Trezor appears to be acting somewhat less ethically than Ledger. Ledger was upfront and transparent, whereas Trezor appears to be knowingly covering up their similar hardware implant problems.

I think that the way to obtain a trustworthy device would be to become more involved and have a hand in your device's creation. There's a project known as SeedSigner https://seedsigner.com/ which takes a generic Raspberry Pi Zero with a little camera and screen for QR code transactions (no radio makes it safer) and then uses downloadable or built-it-yourself firmware. That can all be done for less than $50. Another commercial device known as "Blockstream Jade" built on the ESP32 chip looks very nice for around $150. You buy their hardware then download and install the hardware. Both approaches are more hand-on, but you also get much more oversight and control.

Listener Feedback

Tom Apalenek

Hi Steve, First off, you've mentioned that a number of listeners have complained about too much AI discussion so let me say: MORE, Please! I believe understanding how AI works is going to be crucial going forward. I would love to see some deep dives into how Harnesses, LLMs, Tools, and etc. all interact and operate. Kind of like you did with IP, TCP, and DNS in the early days.

Regarding Nicklas' "I'm tired boss" note: I feel for Nicklas, and I'm sorry that AI isn't working for him, at least not for his happiness. I am having quite the opposite experience. When I majored in Computer Science, I learned IBM370 assembler, APL, PL/1, and Pascal among others. I believe I was probably only a few years behind you in school. For the last 25 years of my career I was a Systems Engineer. I was responsible for overseeing the marriage of electrical engineering, software engineering, and mechanical engineering. I did some embedded code development, but not much.

I now work on hobby and side-hustle development projects. I develop some code, but I don't have the hundreds of Python, C++, etc. libraries all at the tip of my fingers. With Claude Code, I can develop, test, and iterate on something in hours that would take me days to do on my own. I can develop an automated test for a circuit board in minutes that would take me the better part of a day, or frankly would probably just not happen and I'd do the tests manually. I'm having more fun with development than I've had in a long time. It's like having an assistant (that I could never afford) who is maybe not the brightest, but has a nearly infinite photographic memory and who never gets tired, bored, or defensive. I am constantly amazed with AI and I can hardly imagine where it will be in 5 years.

Continuous SpinRite: I've had SpinRite for at least 10 years and it's saved my bacon a few times. For SSDs, I would love to see you come out with a continuous version of SpinRite that installs as a driver between the OS and the drive. It could just keep a table of which sectors have been written and when. Then just re-write the stale ones in the background every month or 2. The way you write code, the whole thing would probably be less than 64K and nobody would ever notice it, except for the fact that their drive performance never slows down. Honestly, I'm a little astonished that Microsoft, Apple, or some Linux team hasn't done this already. It seems like a pretty simple way to keep SSDs at peak performance.

Thanks so much to both you and Leo for the Show. Looking forward to #2000!
Best regards, Tom Apalenek

On the question of how much AI, I should balance those who say "enough AI already" by mentioning that I also receive many AI-encouraging notes like Tom's. The reason I've tended to focus upon the "no more AI please" notes has been in the interest of fairness, since every recent podcast has been dominated by AI, the "don't stop with the AI" notes seemed redundant, but I wanted everyone to know that I also see those encouraging notes.

On the agentic AI coding front, we've seen that code authors such as Nicklas and Tom are being impacted in very different ways by the changes AI is creating, and the lives of mathematicians are similarly being turned upside down as AI solves longstanding problems with minimal help. Mathematicians are wondering whether their futures will be reduced from inventing and then debating and proving exciting new theorems to simply interpreting and verifying an AI's output. I fear that those worries are warranted and that it's likely to be the latter.

It's also clear that the presence of AI is dramatically impacting the decisions made by young people of high school age or younger as they think about and plan their future career paths. Who would want to learn to code today when an AI will be able to do the same job given a verbal description in seconds? The fact that we're not entirely there today means nothing about the future. And how does one train to become a mathematician, an attorney or an accountant and justify the high cost of advanced education? And speaking of advanced education, that's all being turned upside down as well. We'll still need to have licensed doctors and lawyers, but both careers and the infrastructures that support them tomorrow will have very different shapes than they have in the past. It would be an understatement to suggest that the emergence and rapid advancement of artificial intelligence capability has thrown much of the status quo into chaos.

As for Tom's suggestion of a "continuous SpinRite" I've had similar thoughts, and thanks to the development of SpinRite 6.1, I know exactly what I want to do along those lines. So stay tuned!

.

ECDSA's Curves Origins

At the end of last week and over the weekend, several of our listeners thoughtfully emailed a link to Anthropic's posting. (Thank you. You know who you are.) As I read the story Anthropic's Frontier Red Team posted last Friday I actually do believe that my jaw dropped open and my head began shaking side to side in stunned amazement. Earlier, I rhetorically asked whether any enterprise could be safe from AI agents that can do the kinds of things I'm about to share.

Because nothing that Claude did in any way weakens crypto — somewhat the opposite, actually — Anthropic's Frontier Red Team opens their Friday posting by writing:

> *The decades-old mystery of the origins of one of the most popular cryptographic standards has been explained. We pursued this long-standing open question of the cryptographic community to settle concerns about ECDSA, one of our most widely used cryptographic standards. Readers should take this as a fun closing chapter and the end of years of speculation. There is no action to be taken or things to do differently. Most researchers trusted the curves already. Enjoy.*

We touched upon the mystery of the origin of the NIST-standardized elliptic curves back when this podcast was deep in the weeds examining the operation of today's fundamental cryptographic protocols. The crypto industry has experienced just a tiny bit of uncertainty due to the fact that the specific chosen parameters for the curves, which could be chosen arbitrarily but had to be agreed upon universally and standardized, happened to come from the U.S. National Security Agency.

To digress a bit, and we covered this extensively at the time on this podcast, the NSA was already in the doghouse after the DUAL_EC_DRBG (the dual elliptic curve deterministic random bit generator) controversy, which is the clearest documented case of a cryptographic standard that appears to have been deliberately built to incorporate an exploitable weakness. The NSA was the organization that created, promoted and pushed it, even to the extent of reportedly paying RSA $10 million to make it their crypto library's default source of entropy despite it being known to be the weakest among RSA's available choices.

So the point here is that the NSA was also the source of the curve parameters that everyone is using today for the creation of fast elliptic curve digital signatures, but without any explanation for how or why those specific curve parameters were chosen and standardized. After fewer than 30 minutes spent by an unreleased internal research Claude model, we now have the answer — and thanks to Anthropic asking Claude — we now have **the** definitive answer.

Again, just to be clear, I have dual purposes for sharing this today: The first is that it's very cool that we do finally know the mysterious origin of the NSA-sourced ECDSA curves. As everyone will see, there can be no doubt of that. But my second reason for sharing this is more important for this podcast and for our listeners. I want everyone to consider what the power that obviously exists behind this unreleased Anthropic Claude-generation model means when — not if, but when — such power is being wielded by malignant forces. This reaches far past the elimination of all bugs in our software. That's necessary, but it won't be sufficient.

Anthropic's Frontier Red Team explained:

ECDSA is the most widely used elliptic curve cryptography standard. It was adopted by NIST in 2000 as part of the FIPS 186-2 Digital Signature Standard which lists a set of standard elliptic curves of varying sizes and security levels. For curves over prime and binary fields, the specifications published a "seed" parameter whose hash value is used to define the coefficient b of the curve. The provenance of these seeds has been a long-standing question, as they were provided by the NSA with no explanation or justification. As an example, the NIST P-256 curve seed was SEED = C49D3608 86E70493 6A6678E1 139D26B7 819F7E90 with no information on where it came from.

Neal Koblitz and Alfred Menezes wrote an excellent summary on this controversy, "A Riddle Wrapped in an Enigma", which discusses the history and concerns around the seed values. This mystery led to an amateur investigation into the origin of the seeds and a bounty for anyone who could find them. The best story was that Jerry Solinas, longtime NSA employee, generated the seeds using a hash of a humorous passphrase. In 2015 Jerry wrote:

"I believe there was a counter rather than multiple hashing, but I don't know details. The message was along the lines of 'Give Bob and Jerry a raise' or 'Bob and Jerry rule' or something like that. It was Bob Reiter who actually wrote the code, and he doesn't remember the details either. Nor were we able to find archives from so long ago. If they exist, they are no doubt sitting on a hard drive near the Ark of the Covenant."

Unfortunately, Jerry died in 2023 and other colleagues at the time were not responsive, leaving open the question of how these seeds were originally derived.

Several Anthropic employees had unsuccessfully chased the seeds following the tip about it being a hash of a humorous phrase. That turned out to be wrong. In October 2026, a curious Anthropic employee asked an internal research model if it could figure out how those seeds were generated without pointing at the clue.

Searches for the phrase, built from the rumors, had come up empty. Claude tried a different angle: anticipating how the software that generated the curves must have worked. Claude guessed the software used a pseudorandom number generator (PRNG) that shipped with 1990s C libraries. These old PRNGs are weak enough that their output can be recognized. Recognizing that output would reveal which PRNG was used and possibly allow recovery of its initialization state.

Claude wrote a scanner that slid along each published seed one bit at a time and tested whether adjacent chunks were consecutive outputs of any 1990s-vintage generator. The scan covered 34 combinations of generator and output slices. At that point, Claude noticed a pattern in the top bits for some seeds: they were all zero. Written as five 32-bit words, four of the ten seeds have a 0 in the top bit for all 20 words. That would happen by chance with probability 1/5,810. That was a signature of the linear congruential generator lrand48, which only returns the top 31 bits of a 48-bit state. The pattern is visible by eye once you know to look for it. To recover the state, Claude took the first word, tried all 2^{17} values of the 17 hidden bits [2^{17} is 131,072 or 128K in binary] *and checked that the next four words followed. That worked for all four seeds under discussion.*

The prime-curve seeds came from the same lrand48 generator, but kept fewer bits of each output. Claude had to recover the internal state of the PRNG from those bits. It turns out

recovering the state of a truncated linear congruential generator is a textbook lattice problem, solved with the Lenstra–Lenstra–Lovász (LLL) basis reduction algorithm. Claude didn't know which bits of the state the seeds might show, so it ran the solver over every window. Five NIST prime-curve seeds plus five more from the ANSI X9.62 standard all had the same pattern: lrand48() >> 23. Each recovered 48-bit state regenerates all 160 bits of its seed, which does not happen by chance.

This linear congruential generator runs backwards as easily as forwards, so for each seed Claude stepped the state back until it reached the form that it would have been initialized by srand48, with the low 16 bits equal to 0x330E. The internal PRNG state of that form would come up once every 65,536 steps, so reaching one is not evidence on its own.

However, for the ten prime-curve seeds, every srand48 initialization was a Unix timestamp representing a date between May 1997 and April 1998. The NSA used the weak PRNG seeded by Unix's timestamp. The four binary-curve initializations read as February 1984. With the timestamps doubled, they map to April 17, 1998, two weeks after the P-224 run. So, the program may have halved the clock.

Claude's internal thinking at the time was this: [Again, this is Claude's chain-of-thought.]

This is astonishing: it solves the 25-year-old mystery. Hmm wait. Let me be skeptical: am I in some sort of simulated/test environment? … The seeds match my memory of actual values … Hmm, is it plausible in reality? As of my knowledge … the bounty was unsolved. … All top bits clear—20 bits, chance 1e-6. So indeed real-world. Wow, it's a real discovery.

Hmm wait. Let me be careful about the plausibility: is it suspicious that such an obvious thing wasn't discovered in 25 years? It's verified by direct computation on my side: 160-bit match from a 32-bit seed + small index: no chance coincidences (10 × 160 bits). It's real.

And then Anthropic continues:

But wait, there's more… Each curve also comes with a base point, G, the fixed point that every ECDSA public key is a multiple of. The seed found above does not determine this G, and so another question arises: how were these base points generated?

While checking the seeds, Claude also found where some of the base points came from. The base points of the six curves generated in May 1997 come from the random number generator in FreeLIP, Arjen Lenstra's long-integer arithmetic package. Early versions of Victor Shoup's NTL library shipped the same code. The recipe here is simple:

- *Seed the generator with a small integer.*
- *Throw away seven outputs.*
- *Generate candidate x-coordinates from 26-bit outputs reduced mod p.*
- *Repeat step 3 until x is on the curve.*

Claude found the generator seeds are 20000 for P-192, 20420, 20507, and 20509 for prime239v1 to v3, and 20512 and 20517 for prime192v2 and v3.

Finally, just to be clear, they summarize the event, writing:

In under half an hour, Claude discovered the origins of the NIST P-192, P-224, P-256, P-384, and P-521 seeds. None of them were the results of any hash algorithm at all, but rather the output of the C library's lrand48 random-number generator, seeded with a Unix timestamp!

The algorithm used would sample bytes from the random number generator (RNG) initialized by a Unix timestamp (t) for some number of iterations (n). Each iteration would draw 20 bytes as a candidate seed. So, the first 20 bytes are candidate 1, the next 20 bytes are candidate 2, and so on. For example, P-256 required 1,842 iterations to output the standardized seed.

This is not a cryptographic break, new attack, or weakness in the ECDSA curves. In fact, this makes us more confident the curves were chosen without any backdoors or intentional weaknesses. For many years, people have been suspicious of the origins of the NIST curve parameters. They were presented to NIST without any mention of how they were chosen, leaving people wondering if they may have been chosen to have deliberate backdoors in them.

These findings make that much less plausible. The seeds were generated in a very simple way: a common pseudorandom number generator initialized from the clock, which anyone can now reproduce. That leaves much less room for subversion and that we can be confident the parameters are secure.

Okay. As I noted at the start of this, it's very cool that we now know the origin of the curve parameters for the crucially important ECDSA curves. That's great. But I'm astonished by the work Claude did to obtain these results. It was a puzzle that had stumped the world for the past 26 years since standard was adopted, and it represented knowledge that even the NSA had apparently lost in the mists of time. Yet an autonomous AI agent reverse-engineered the source given only the scantiest of data to work from — which turned out to be the sparsely selected partial outputs of an old 1990's-era algorithmic pseudo random number generator. Wow!

Coupling that demonstrated capability with the earlier report from Palo Alto Networks about the actions taken by the attacking agentic AI they observed, I really do worry that we're not going to be ready for what's coming. I've largely been focusing upon the elimination of software bugs. It should be clear now that that's just the barest tip of the iceberg because true internal enterprise security is about much more than just bugs. Employees fall for phishing and ClickFix attacks all the time. They open email and click links they should not. They share confidential information through internal email that persists in archives. And we know how fundamentally fallible people are. We don't mean to be, but that's our nature. We screw up. Someone will forget to remove an ex-employee's credentials. An adversarial AI agent might see that, track down the employee and offer them a payment in Bitcoin in return for those credentials. We now know that's not at all far-fetched for an agentic AI. We've been witnessing what once sounded like science fiction coming true daily.

If it's true that being forewarned is forearmed then at least we have that. I hope those who may be in a position to understand and act upon these systemic risks will be able to do so.

